

S o C

Separation of Concerns

Principle

Different concerns, different modules

DEFINITION

Split a system so each part addresses a distinct concern — UI, domain, persistence.

A "concern" is one dimension of a problem — presentation, domain rules, persistence, plus cross-cutting ones like logging and auth. Separation of Concerns puts each in its own module or layer so it can be understood, changed and tested on its own. It is the same instinct as SRP, scaled up from a single class to the architecture of the whole system.

WHY IT MATTERS

When concerns mix — SQL in the controller, business rules in the view — every change reaches into unrelated code, nothing can be tested in isolation, and ownership blurs. Clean seams let you swap the database, redesign the UI, or change a rule without disturbing the others, and they let different people work in parallel without colliding. It is what keeps a codebase navigable as it grows.

— How it works

Concern	One dimension of the problem: presentation, domain logic, persistence, or a cross-cutting aspect like logging or auth.
Layer / module	The unit that owns exactly one concern and exposes a thin contract to the next — controller, service, repository.
Boundary	The interface between concerns. Dependencies cross it in one direction (toward the domain), so each side can change behind it.

HOW TO APPLY

1. Name the concerns in the slice you are building: HTTP, rules, storage, cross-cutting.
2. Give each its own module and let them talk through thin contracts.
3. Point dependencies toward the domain (DIP), keeping infrastructure at the edges.
4. Pull cross-cutting concerns (logging, auth, transactions) into middleware or decorators, not business code.

LITMUS TEST

Can you change the database, the UI, or a business rule without touching the other two? If one edit ripples across all layers, the concerns aren't separated.

— Smell → fix

One handler mixes HTTP, business rules and SQL:

```
function handler(req, res) {  
  let total = 0  
  for (const i of req.body.items) total += i.price // domain rule  
  db.query('INSERT INTO orders ...') // persistence  
  res.render('receipt', { total }) // presentation  
}
```

↓ SPLIT CONCERNS INTO LAYERS

```
// controller → service (domain) → repository (persistence)  
class OrdersController { create(req){ this.svc.place(req.body) } }  
class OrderService { place(o){ /* business rules */ } }  
class OrderRepository { insert(o){ /* SQL only */ } }
```

Each layer owns one concern and talks to the next through a thin contract. Swap the database or the UI without touching domain rules.

USE WHEN

- ✓ Layered or hexagonal architecture
- ✓ UI logic leaking into the domain

AVOID WHEN

- ▲ Over-layering tiny apps — ceremony without benefit

— Trade-offs & pitfalls

PROS

- + Independent change and testing
- + Clear ownership

CONS

- More indirection
- Cross-cutting concerns still leak

COMMON PITFALLS

- ▲ Over-layering a small app — ceremony and indirection with no payoff (KISS / YAGNI).
- ▲ Leaky layers: ORM entities or HTTP types bleeding into the domain, re-coupling what you split.
- ▲ Scattering a cross-cutting concern (logging, auth) through every layer instead of isolating it.

WHERE YOU SEE IT

- A NestJS controller → service → repository, each owning one concern.
- Hexagonal / clean architecture with the domain at the center and adapters at the edges.
- Frontend split: presentation components vs state/data services (Angular) or hooks (React).

SEE ALSO

SRP	the same idea at class scale — SoC is its architectural scale
MVC	a classic SoC split of input, output and model
DIP	keep layer dependencies pointing at abstractions, not details
DRY	one concern per module gives each rule a single home

— Interview & sources

SoC vs SRP?

Same instinct, different scale. SRP is one reason to change for a class; SoC is one concern per module or layer for a whole system.

How do cross-cutting concerns fit?

Logging, auth and transactions cut across layers; isolate them with middleware, decorators or AOP instead of scattering them through business code.

Can you over-separate?

Yes — layering a tiny app adds ceremony with no payoff. Match the number of layers to the real complexity.

SoC vs modularity vs cohesion?

SoC decides what to separate (by concern); cohesion describes a module that keeps one concern's parts together; modularity is the mechanism of boundaries that enforces it. They reinforce each other.

REMEMBER

Give each distinct concern its own module so it can change on its own.

SOURCES

Edsger W. Dijkstra — "On the role of scientific thought" (1974): the term separation of concerns

Robert C. Martin — "Clean Architecture" (2017): drawing boundaries between concerns